

Topics to cover -

1. [Process Management](#)
 - a. Process creation
 - b. States of a process
 - c. Inter process communication (IPC)
 - d. Vulnerabilities of IPC
2. [CPU Scheduling](#)
 - a. FCFS
 - b. SJF
 - c. Priority
 - d. Round Robin
3. [Deadlocks](#)
 - a. Conditions
 - i. Mutual Exclusion
 - ii. Hold and Wait
 - iii. No Preemption
 - iv. Circular Wait
 - b. Detection
 - i. **Resource Allocation Graph (RAG)**
 - ii. **Wait For Graph (WAG)**
 - c. Avoidance - Banker's Algorithm
4. [Memory Management](#)
 - a. Memory allocation techniques
 - i. Fixed memory partition
 1. First fit
 2. Best fir
 3. Worst fit
 - ii. Variable memory partition
 1. Paging
 2. Segmentation
 - b. Fragmentation
 - i. Internal
 - ii. External
 - c. Page fault handling technique
 - d. Page allocation algorithms
 - i. LRU
 - ii. Optimal
 - iii. FIFO
 - e. Logical and Physical address related maths
 - f. Linux commands

Process Management

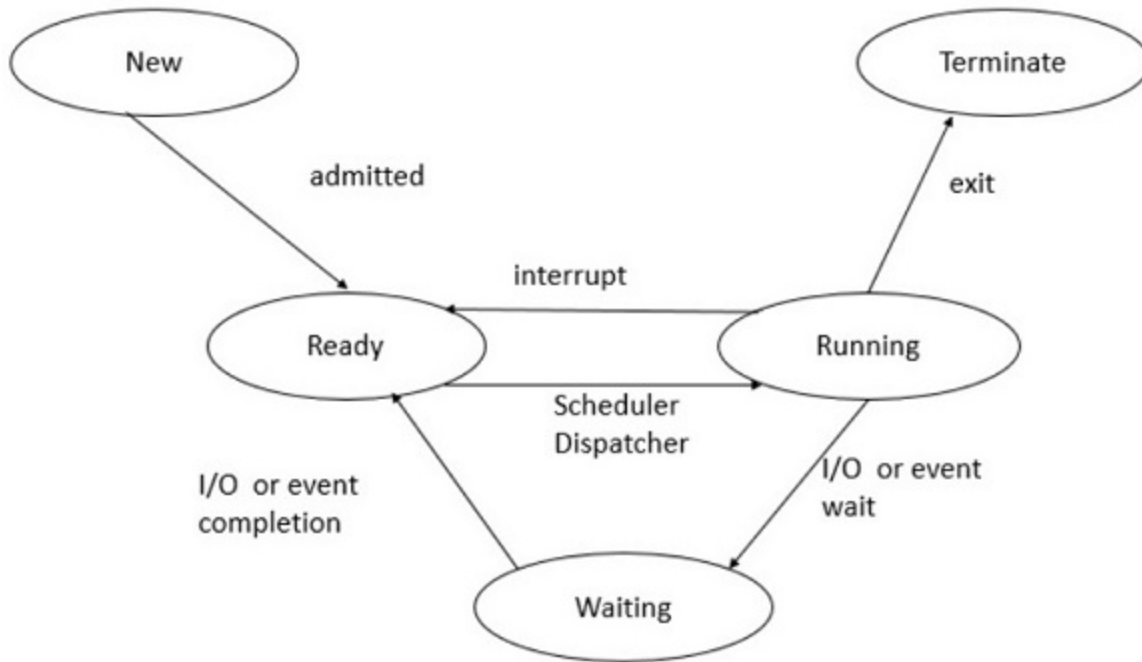
In an operating system, a **program** is a passive set of instructions written using a programming language to perform a specific task and stored on a disk or secondary memory.

A **process** is an instance of a program while it's running.

A **thread** is the smallest unit of execution within a process, often called a lightweight process.

Process Creation - Every new process is created by an already running process using system calls, making the creator the parent and the newly formed one the child.

States of a process -



A race condition vulnerability occurs when two or more processes or threads access shared resources simultaneously without proper synchronization, causing the program's behavior to depend on the order of execution. Such vulnerabilities can lead to data corruption, privilege escalation, inconsistent program states, and security breaches.

A critical section is a part of a program where shared resources (like memory, files, or variables) are accessed by multiple processes or threads. To avoid problems such as race conditions and data inconsistency, only one process/thread should execute the critical section at a time using synchronization techniques.

The solution to critical section or race condition vulnerability is locking the processes during execution.

CPU Scheduling

★ CPU SCHEDULING ALGORITHMS

FCFS (FIRST COME, FIRST SERVE)

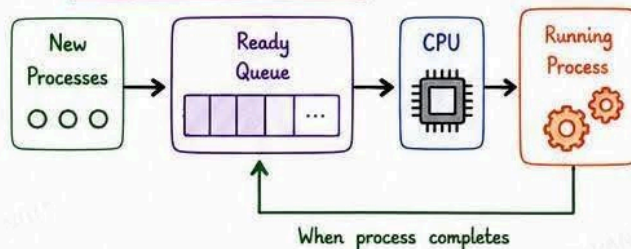
★ CONCEPT

- FCFS (First Come, First Serve) is the simplest CPU scheduling algorithm.
- Processes are executed in the order in which they **arrive** in the Ready Queue.
- It is a **Non-Preemptive** algorithm.



Once a process gets the CPU, it runs until it completes (no interruption).

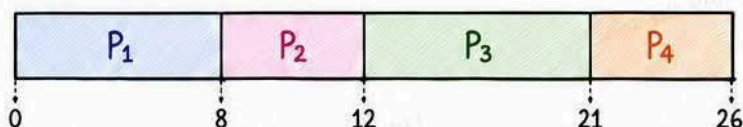
★ FCFS - EXECUTION FLOW



★ EXAMPLE

Process	Arrival Time (AT)	Burst Time (BT)
P ₁	0	8
P ₂	1	4
P ₃	2	9
P ₄	3	5

① GANTT CHART (FCFS)



Order of execution: P₁ → P₂ → P₃ → P₄

② CALCULATION OF CT, TAT & WT

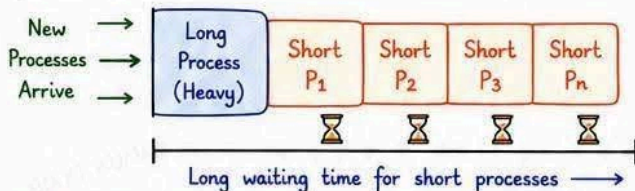
Process	AT	BT	Completion Time (CT)	Turnaround Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P ₁	0	8	8	8 - 0 = 8	8 - 8 = 0
P ₂	1	4	12	12 - 1 = 11	11 - 4 = 7
P ₃	2	9	21	21 - 2 = 19	19 - 9 = 10
P ₄	3	5	26	26 - 3 = 23	23 - 5 = 18

$$\text{Average Waiting Time (AWT)} = (0 + 7 + 10 + 18) / 4 = 8.75$$

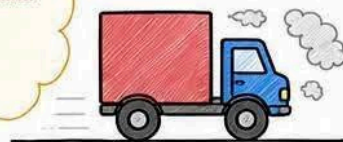
$$\text{Average Turnaround Time (ATAT)} = (8 + 11 + 19 + 23) / 4 = 15.25$$

★ CONVOY EFFECT (Drawback of FCFS)

A long process at the front of the queue delays the shorter processes behind it.



Short processes wait for a long time behind the long process. This is called **Convoy Effect**.



★ ADVANTAGES

- ✓ Very simple and easy to implement.
- ✓ Fair - processes are executed in order of arrival.
- ✓ No starvation (every process will get CPU).
- ✓ Low scheduling overhead.



★ DISADVANTAGES

- ✗ Poor average waiting time.
- ✗ Convoy effect occurs.
- ✗ Not suitable for systems with mixed long and short processes.



★ EXAM / INTERVIEW KEY POINTS

- FCFS is the simplest CPU scheduling algorithm.
- It is Non-Preemptive.
- Processes are executed in the order of their arrival.
- Easy to implement but average waiting time is high.

- Convoy effect is a major drawback of FCFS.
- Suitable for batch systems where all jobs are of similar length.



★ PRIORITY SCHEDULING

★ CONCEPT

- In **Priority Scheduling**, each process is assigned a **Priority**.
- The process with the **Highest Priority** gets the CPU first.
- If two or more processes have same priority, **FCFS** is used among them.
- Priority Scheduling can be of two types :
 - **Non-Preemptive Priority Scheduling**
 - **Preemptive Priority Scheduling**



Note : Here we consider smaller number = Higher Priority (1 is highest priority).

★ EXAMPLE

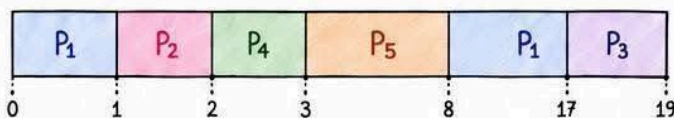
Process	Arrival Time (AT)	Burst Time (BT)	Priority (Smaller = Higher)
P ₁	0	10	3
P ₂	1	1	1
P ₃	2	2	4
P ₄	3	1	2
P ₅	4	5	2

Selection Rule

Choose the process with highest priority (smallest number).
If tie → FCFS



★ GANTT CHART (Preemptive Priority Scheduling)



Step by Step Selection (why above order?)

- t=0 :** Only P₁ is available → Run P₁
- t=1 :** P₂ arrives (Priority 1 < 3) → Preempt P₁, Run P₂
- t=2 :** Ready {P₁(rem 9), P₃(4)} → Highest Priority P₄ (2) at t=3
- t=3 :** P₄ runs (BT=1) and completes
- t=4 :** P₅ arrives (Priority 2). Ready {P₁(9), P₅(2), P₃(4)}
→ P₅ has higher priority → Run P₅ till t=8
- t=8 :** Ready {P₁(9), P₃(4)} → Run P₁ till t=17
- t=17 :** Only P₃ left → Run P₃ till completion

★ CALCULATION OF CT, TAT & WT

Process	AT	BT	Priority	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P ₁	0	10	3	17	17 - 0 = 17	17 - 10 = 7
P ₂	1	1	1	2	2 - 1 = 1	1 - 1 = 0
P ₃	2	2	4	19	19 - 2 = 17	17 - 2 = 15
P ₄	3	1	2	3	3 - 3 = 0	0 - 1 = -1 → 0
P ₅	4	5	2	8	8 - 4 = 4	4 - 5 = -1 → 0

Note : In WT column, if value is negative, we consider it as 0 because waiting time cannot be negative.

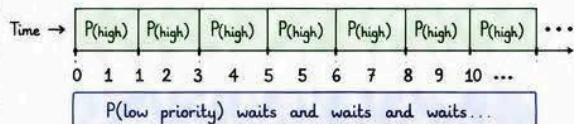


$$\begin{aligned} \text{Average Waiting Time (AWT)} &= (7 + 0 + 15 + 0 + 0) / 5 \\ &= 22 / 5 = 4.4 \text{ ms} \end{aligned}$$

$$\begin{aligned} \text{Average Turnaround Time (ATAT)} &= (17 + 1 + 17 + 0 + 4) / 5 \\ &= 39 / 5 = 7.8 \text{ ms} \end{aligned}$$

★ STARVATION PROBLEM

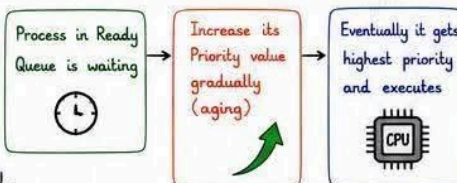
If high priority processes keep arriving continuously, low priority processes may wait for a very long time.



This may lead to Starvation of low priority process. 😞

★ AGING TECHNIQUE (To prevent starvation)

In aging, waiting process is gradually given higher priority so that it will eventually get CPU.



Aging helps in fairness and avoids starvation. 😊

★ PREEMPTIVE vs NON-PREEMPTIVE PRIORITY SCHEDULING

Preemptive Priority Scheduling	Non-Preemptive Priority Scheduling
- If a new process arrives with higher priority than the running process, the running process is preempted. - Better response time. - More overhead (due to context switching).	- Once a process gets CPU, it runs till completion or blocks for I/O. - Simpler and less overhead. - Poor response time for high priority processes.

★ ADVANTAGES

- ✓ Important jobs get CPU first.
- ✓ Good for real-time systems.
- ✓ Flexible (both preemptive and non-preemptive).

★ DISADVANTAGES

- ✗ Starvation of low priority processes.
- ✗ Hard to choose priority values.
- ✗ May lead to priority inversion.

★ EXAM / INTERVIEW KEY POINTS

- Priority Scheduling uses priority to decide which process runs next.
- Smaller number means higher priority (in this note).
- Can be Preemptive or Non-Preemptive.
- Tie in priority → FCFS is used.
- Starvation can occur for low priority processes.

- Aging is used to prevent starvation.
- Useful in systems where some jobs are more important than others.
- Priority inversion may occur in some cases.
- Widely used in real-time and batch systems.



★ CPU SCHEDULING ALGORITHMS

SJF – SHORTEST JOB FIRST (NON-PREEMPTIVE)

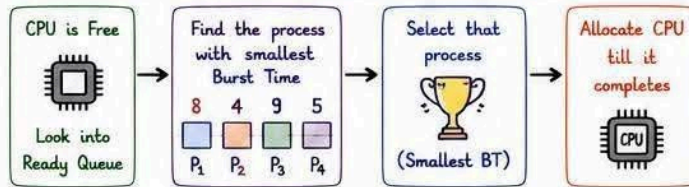
★ CONCEPT

- SJF (Shortest Job First) selects the process with the **smallest Burst Time** from the Ready Queue.
- It is a **Non-Preemptive** algorithm.
- When CPU becomes free, the shortest job among available processes is executed.



Goal → Reduce average waiting time.

★ HOW SJF WORKS ? (READY QUEUE SELECTION)

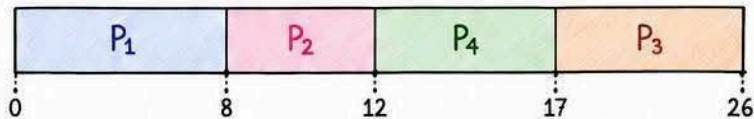


If two or more processes have **same Burst Time**, select the one which arrived first. (1)

★ EXAMPLE

Process	Arrival Time (AT)	Burst Time (BT)
P ₁	0	8
P ₂	1	4
P ₃	2	9
P ₄	3	5

★ GANTT CHART (SJF)



Order of execution (by SJF):
P₁ → P₂ → P₄ → P₃

At time 0 → only P₁ is available.
After P₁ completes → P₂ (4) is shortest.
Then P₄ (5) is shortest.
Finally P₃ (9) runs.

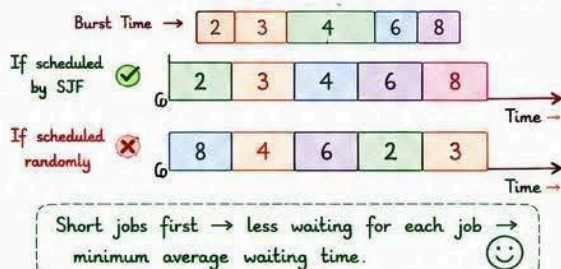
★ CALCULATION OF CT, TAT & WT

Process	AT	BT	Completion Time (CT)	Turnaround Time (TAT) = CT - AT	Waiting Time (WT) = TAT - BT
P ₁	0	8	8	8 - 0 = 8	8 - 8 = 0
P ₂	1	4	12	12 - 1 = 11	11 - 4 = 7
P ₄	3	5	17	17 - 3 = 14	14 - 5 = 9
P ₃	2	9	26	26 - 2 = 24	24 - 9 = 15

$$\text{Average Waiting Time (AWT)} = (0 + 7 + 9 + 15) / 4 = 7.75$$

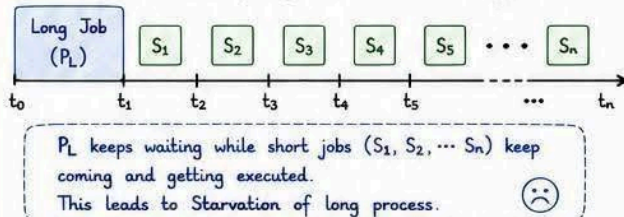
$$\text{Average Turnaround Time (ATAT)} = (8 + 11 + 14 + 24) / 4 = 14.25$$

★ WHY SJF GIVES MINIMUM AVERAGE WAITING TIME ?



★ STARVATION PROBLEM IN SJF

If short jobs keep arriving continuously, long job may have to wait for a very long time.



★ ADVANTAGES

- ✓ Gives minimum average waiting time.
- ✓ Better response for short jobs.
- ✓ High throughput (more jobs completed per unit time).
- ✓ Good for batch systems where short jobs are common.

★ DISADVANTAGES

- ✗ Starvation of long processes.
- ✗ Requires knowledge of Burst Time in advance (not practical).
- ✗ Not suitable for interactive/time-sharing systems.

★ FCFS vs SJF (QUICK COMPARISON)

Feature	FCFS	SJF (Non-Preemptive)
Selection	By Arrival Time	By Shortest Burst Time
Preemption	No	No
Average Waiting Time	Higher	Lowest
Starvation	No	Possible
Implementation	Very Simple	Comparatively Complex
Best for	Batch systems with similar jobs	Batch systems with varying jobs

★ EXAM / INTERVIEW KEY POINTS

- SJF selects the process with the smallest Burst Time from Ready Queue.
- It is Non-Preemptive.
- It gives minimum average waiting time among all non-preemptive scheduling algorithms.
- Needs knowledge of Burst Time in advance.

- Starvation may occur for long processes.
- If two processes have same Burst Time, choose the one with earlier arrival.
- Used in batch systems, not ideal for time-sharing systems.



★ ROUND ROBIN (RR) SCHEDULING

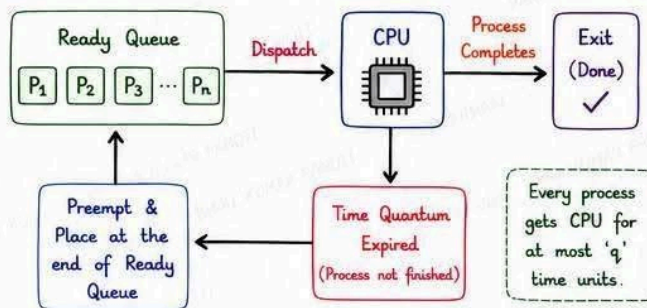
★ CONCEPT

- Round Robin is a **Preemptive** CPU scheduling algorithm.
- Each process gets a fixed amount of CPU time called **Time Quantum** or **Time Slice** (q).
- If a process does not finish within this time, it is **preempted** and placed at the end of the **Ready Queue**.
- If it finishes before the quantum, it leaves the system.



Goal → Give fair chance to every process and good response time.

★ RR WORKING (CIRCULAR READY QUEUE)

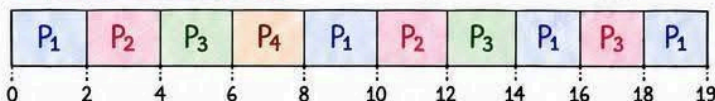


★ EXAMPLE

Process	Arrival Time (AT)	Burst Time (BT)	Time Quantum (q)
P ₁	0	10	2 (fixed)
P ₂	1	4	2
P ₃	2	5	2
P ₄	3	3	2

All times are in milliseconds (ms).

★ GANTT CHART ($q = 2$)



Step by Step Execution

- $t=0$: P₁ runs (0-2)
 - $t=1$: P₂ arrives
 - $t=2$: P₃ arrives, P₁ → end
 - $t=3$: P₄ arrives
 - $t=4$: P₂ → end
 - $t=6$: P₃ → (rem 3)
 - $t=8$: P₄ → (rem 1)
 - $t=10$: P₁ → (rem 8)
 - $t=12$: P₃ → (rem 1)
 - $t=14$: P₁ → (rem 6)
 - $t=16$: P₃ → end
 - $t=18$: P₁ → end
- All processes completed at $t = 19$

★ CALCULATION OF CT, TAT & WT

Process	AT	BT	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P ₁	0	10	19	$19 - 0 = 19$	$19 - 10 = 9$
P ₂	1	4	6	$6 - 1 = 5$	$5 - 4 = 1$
P ₃	2	5	16	$16 - 2 = 14$	$14 - 5 = 9$
P ₄	3	3	10	$10 - 3 = 7$	$7 - 3 = 4$

Average Waiting Time (AWT)
 $= (9 + 1 + 9 + 4) / 4$
 $= 23 / 4 = 5.75 \text{ ms}$

Average Turnaround Time (ATAT)
 $= (19 + 5 + 14 + 7) / 4$
 $= 45 / 4 = 11.25 \text{ ms}$

★ EFFECT OF TIME QUANTUM (q)

Very Large Quantum
 $(q \geq \max \text{BT})$

Optimal Quantum

Very Small Quantum
 $(q \rightarrow \text{very small})$



RR behaves like FCFS
 (No preemption effectively).
 Waiting time increases.



Good balance between
 response time and
 context switching.



Very frequent context
 switching occurs.
 Overhead becomes high.

Choose q carefully depending on system & workload. 😊

★ STARVATION PROBLEM

Low priority (or less CPU time) processes may wait for a very long time.

High priority / Short jobs keep coming...



Low priority job (L)

L keeps waiting and may never get CPU. 😞

★ AGING TECHNIQUE (To prevent starvation)

Waiting process priority is gradually increased with time.

Process is waiting in Ready Queue

Increase its Priority value gradually (Aging)

Eventually it gets higher priority and executes on CPU



★ RR vs FCFS (Quick comparisons)

Feature	FCFS	Round Robin (RR)
Type	Non-Preemptive	Preemptive
Execution	Runs till completion	Runs for 'q' time
Response Time	Poor for long jobs	Good for all jobs
Fairness	Not fair	Fair (every process gets CPU)
Overhead	Low	Higher (context switching)
Best Suitable For	Batch systems	Time-sharing systems

★ ADVANTAGES

- ✓ Fair to all processes.
- ✓ Good response time for users.
- ✓ Prevents one process from monopolizing the CPU.
- ✓ Suitable for time-sharing systems (e.g., OS, Interactive systems).



★ DISADVANTAGES

- ✗ More context switching → more overhead.
- ✗ Performance depends on Time Quantum.
- ✗ If q is too small, CPU time wasted in switching.
- ✗ If q is too large, behaves like FCFS.



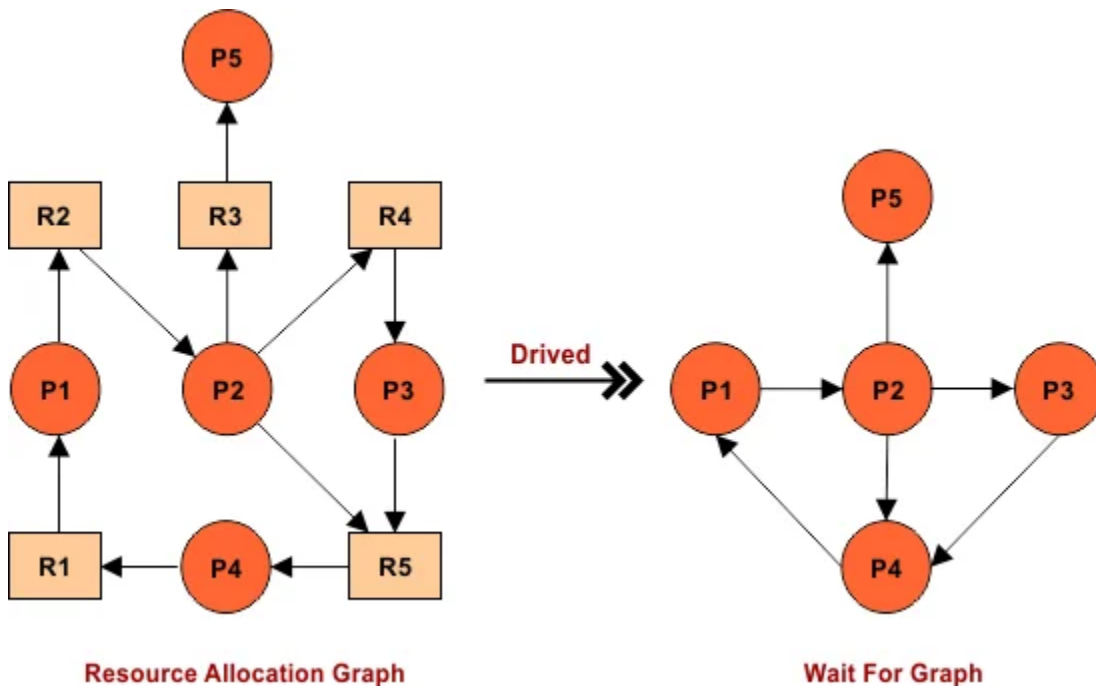
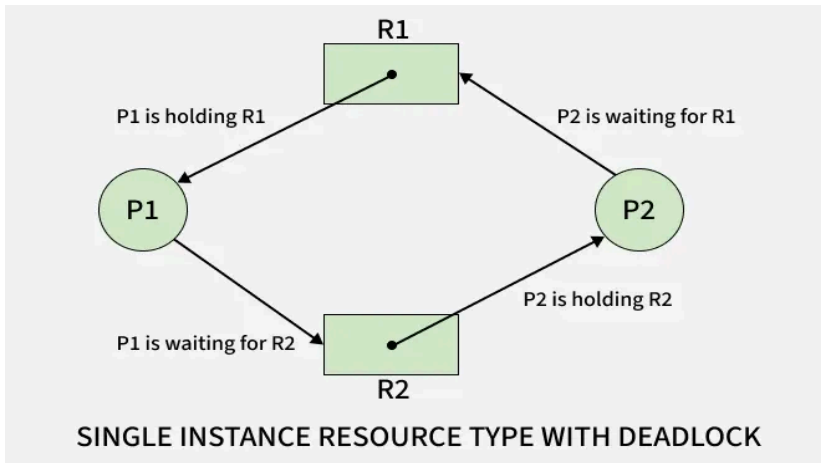
★ EXAM / INTERVIEW KEY POINTS

- RR is a Preemptive scheduling algorithm.
- Uses Time Quantum (q) or Time Slice.
- After q time units, process is preempted if not completed.
- Incomplete process goes to the end of Ready Queue.

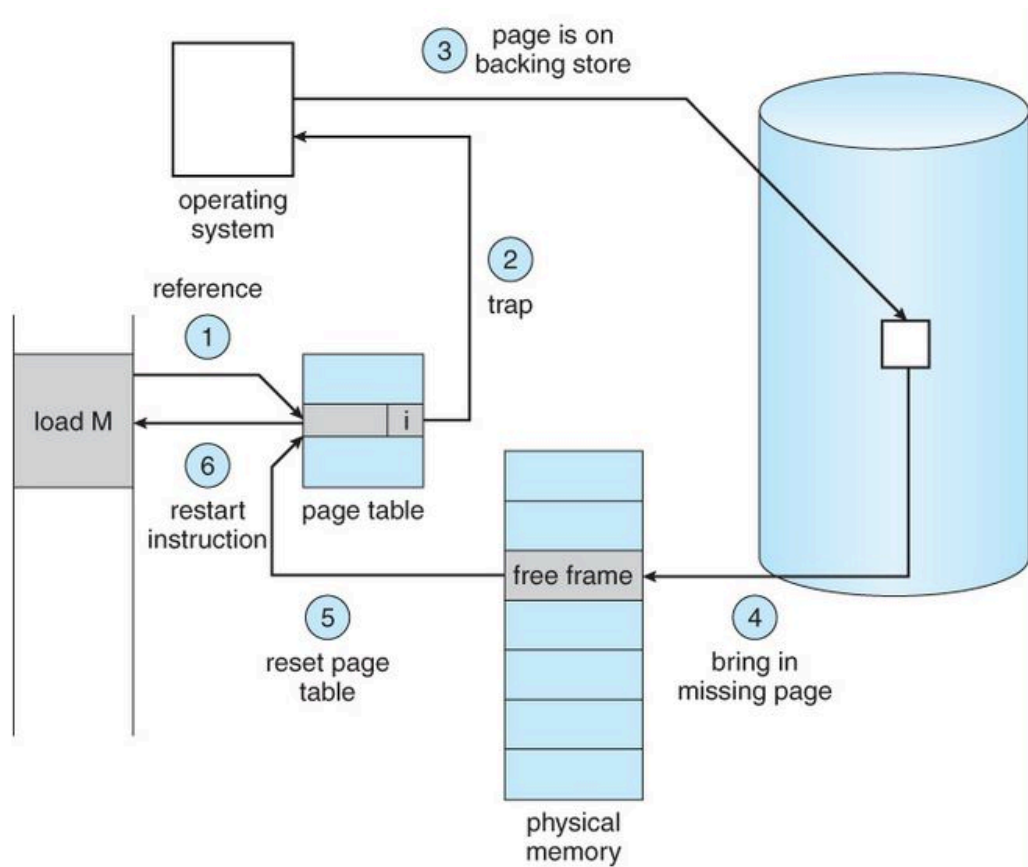
- Good for interactive and time-sharing systems.
- Performance depends on proper choice of quantum.
- Very small q causes high context switching.
- Very large q makes RR similar to FCFS.



Deadlocks



Memory Management



Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Figure - Optimal page replacement algorithm

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Here LRU has same number of page fault as optimal but it may differ according to question.

Figure - Least recently used page replacement algorithm